

# Emil Senior QA Automation — Interview Q&A Prep

Panel: Thomas Provoost (EM) · Hussein Farhat (Staff Eng) · Dmytro Nechyporenko (CTO) — 90 min.

How to use: read each question, **try answering out loud first**, then check. The "Follow-up" notes are the pushback they're likely to apply — rehearse those too. Don't memorize verbatim — own the *\*reasoning\** so you can pivot.

## A. Opening — your 60-second pitch

### A1. "Walk us through what you built."

"An OpenAPI 3.0 spec for a Claims API, a Playwright API-test suite that exercises it, an in-memory Express mock that implements the spec faithfully — including the state machine and business invariants — and a GitHub Actions pipeline. The spec is the single source of truth: `lib/openapi.ts` loads the YAML once at boot and gives the same compiled AJV validators to both the mock (to reject bad requests) and the test suite (to assert response shapes). That means schema drift between the spec and either side is impossible — both fail together. 41 tests, four spec files, organized by API operation. Negative cases are data-driven so adding a new validation rule is one line in an array, not a new test function."

**Follow-up:** *"Why a mock at all?"* → "Because the brief said test cases come from the spec; without a backend I needed a faithful implementation to assert against. The mock is ~200 lines and lets me model what `json-server` can't — the state machine and the payout invariants. Same suite runs against a real server by setting `BASE_URL`."

## B. OpenAPI / Spec design

### B1. "Why OpenAPI 3.0 as source of truth?"

"It's machine-readable and tool-rich — AJV consumes it for runtime validation, Swagger UI renders it, contract-test tools (Pact, schemathesis) and codegen tools all speak it. Writing the contract first means the spec, the mock, and the tests can never drift silently — any change shows up in three places."

### B2. "Why `additionalProperties: false` everywhere?"

"Strict schemas catch two real bugs: clients accidentally relying on server-ignored fields, and silent typos like `policy_number` instead of `policyNumber` returning 200 with the field dropped. The cost is breaking-change risk if we add a new field — but additive changes go to clients, not from them, so this is the right default for *\*requests\**. For *\*responses\**, strict mode catches contract drift on our side."

**Follow-up:** *"Doesn't this hurt forward compatibility?"* → "For request bodies, yes — by design. Clients shouldn't send unknown fields. For responses, OpenAPI consumers should ignore unknown fields anyway; strict response validation lives only in the test suite, not in client code."

### B3. "Why `CreateClaimRequest` and `Claim` are separate schemas."

"Different shapes. `Claim` (response) has `id`, `createdAt`, `updatedAt`, `status` — server-controlled. `CreateClaimRequest` has only client-supplied fields. Treating them as one schema means clients can send `id` or `createdAt` and the server has to strip them. Splitting makes the contract enforce 'server mints these.'"

### B4. "Why is `status` on `CreateClaimRequest` an enum of just `[OPEN]`?"

"Some teams write integration code that always sends `status` for symmetry with response bodies. Letting them send `OPEN` keeps that pattern safe; rejecting any other value forces them through `PATCH` for real

workflow changes. It's an explicit affordance — the schema documents that workflow happens elsewhere."

#### **B5. "Why minProperties: 1 on UpdateClaimRequest?"**

"Empty PATCH is a client bug. Without this, PATCH {} would touch updatedAt and return 200, which is misleading."

#### **B6. "What's wrong with the policyNumber regex ^POL-[0-9]{4,10}\$?"**

"It's a placeholder shape — real Emil policy numbers might use letters, country codes, or check digits. I picked a pattern restrictive enough to test pattern-matching code paths and obvious enough that anyone reading it knows it's mock data. In production, this comes from the policy service's spec."

#### **B7. "Why limit Currency to four codes?"**

"EMIL is European auto insurance, so EUR/CHF/GBP cover most cases and USD covers the edge. Open-ended ISO-4217 enums make negative tests harder — you'd need a denylist of unsupported codes. Constraining the set documents what payout settlement actually supports."

#### **B8. "Walk me through your Error schema."**

"Three fields: code (stable machine-readable, like INVALID\_STATUS\_TRANSITION), message (human prose), and optional details array for field-level validation errors with path and issue. The split matters: clients branch on code, support reads message, and details lets us tell a frontend exactly which field to highlight. Every 4xx response on the API matches this shape — assertions in TC-C9 and TC-U4d enforce that with validators.Error."

### **C. Mock server design**

#### **C1. "Why Express and not Fastify, Hono, Koa?"**

"Boring is good for a mock. Express is the most familiar Node HTTP layer in the industry — anyone reviewing this can read it without ramp-up. Performance doesn't matter for a test mock; readability does. If this were a real service Fastify or Hono would be a serious option."

#### **C2. "Why an in-memory mock instead of json-server / Mockoon / Prism?"**

"The PDF explicitly names invalid status transitions as a negative test case. json-server doesn't model state machines — it's a CRUD passthrough. Prism (the Stoplight mock) does spec-driven mocking but can't enforce business rules like payoutAmount required on APPROVED. The Express mock is ~200 lines and gives me a faithful enough implementation that the test suite exercises real behavior, not tautologies."

**Follow-up:** *"What's the risk of that approach?"* → "The mock is implementing the contract twice — in code and in YAML. If the real backend behaves differently from my mock, the suite passes locally but fails against staging. Mitigation: AJV validators come from the YAML, not the mock, so contract-shape drift fails immediately. Behavior drift (different status codes, different transition rules) only surfaces when you point at the real service — which `BASE_URL=...` lets you do without changing a line of test code."

#### **C3. "Why export app separately from app.listen()?"**

"Two reasons. First, Playwright's webServer block needs a path it can spawn with a custom PORT. Second, if I ever want to run integration tests in-process — pass app directly to supertest for example — I don't need to bind a real port. Routing logic and network binding are decoupled. It's a one-line cost for both options."

#### **C4. "Why is the request logger opt-in?"**

"Test runs need clean output — pass/fail is what reviewers read. When debugging locally I want full request/response bodies. LOG\_REQUESTS=true npm run start turns it on, default off. Same logic as trace: 'retain-on-failure' in Playwright config — instrumentation when you need it, silence when you don't."

## C5. "What invariants does the mock enforce that aren't in the OpenAPI schema?"

"Three. (1) damageDate must not be in the future → 422 DAMAGE\_DATE\_IN\_FUTURE. (2) APPROVED/PAID require payoutAmount → 422 PAYOUT\_REQUIRED. (3) Payout fields are rejected before APPROVED → 422 PAYOUT\_NOT\_ALLOWED. The schema enforces \*shape\*; these are \*business rules\* — well-formed payloads that violate domain semantics. Hence 422, not 400."

## C6. "Why does payoutCurrency default to EUR but only when payoutAmount is set?"

"EUR is Emil's home currency. Defaulting prevents an inconsistent state where amount is present but currency is null. The check `claim.payoutAmount !== undefined && claim.payoutCurrency === undefined` only fires when the test/client explicitly set the amount — it never fabricates a currency on a claim that has no payout."

## C7. "Show me the state-machine code."

"[src/claims-mock-app.ts:45](src/claims-mock-app.ts:45). It's a `Record<ClaimStatus, ClaimStatus[]>` lookup table — OPEN: ['IN\_REVIEW'], IN\_REVIEW: ['APPROVED', 'REJECTED'], APPROVED: ['PAID'], REJECTED and PAID empty arrays meaning terminal. `canTransition(from, to)` is a one-line membership check. Adding a new status is one entry; TypeScript's `Record<ClaimStatus, ...>` forces exhaustiveness so a new status without an entry is a compile error."

## D. State machine deep-dive

### D1. "Why are REJECTED and PAID terminal? What about reopening a rejected claim?"

"Spec scoping decision. Real claim systems do reopen — usually 'REJECTED → IN\_REVIEW' under a supervisor role within an SLA. I called that out in the README's 'What else I'd test' section. For the take-home, terminal-only keeps the state machine teachable and makes the negative cases unambiguous. Production: I'd model it as REJECTED → IN\_REVIEW guarded by role and reopenedAt timestamp."

### D2. "Why allow OPEN → OPEN as a no-op?"

"Idempotency. A retried PATCH from a network blip shouldn't fail. The check `status !== claim.status` skips the transition guard when target equals current. The cost is that `payoutTouched` invariants still apply — so PATCH {status: 'OPEN', payoutAmount: 100} correctly returns 422 even though the status is unchanged."

### D3. "Why 4 transition patterns instead of all 15 forbidden pairs?"

"Signal-to-noise. The mock implements `canTransition` as a lookup table — every forbidden pair fails through the \*same\* code path. Testing all 15 means 15 tests that all fail or all pass together; one bug, fifteen red. I picked four representative \*patterns\* — skip a step, skip to terminal, go backwards, escape a terminal — because each represents a distinct \*kind\* of client mistake. If a real backend with hand-rolled if/else replaces the mock, exhaustive parameterization is one loop away."

**Follow-up:** "I disagree — I'd want exhaustive coverage." → "Fair. The right answer depends on how the production service implements transitions. If it's a single lookup like mine, four is enough. If it's hand-coded per-status logic (as legacy services often are), exhaustive matters because each pair has its own branch. I'd parameterize when migrating to the real backend."

#### D4. "Why does TC-U4d go all the way to PAID before testing the terminal-state escape?"

"Because PAID is the most common terminal a real claim reaches — REJECTED is the dead-end branch. Testing the most likely real-world claim shape catches the bug closest to where users would hit it. Also: it exercises the full happy path on the way there, giving incidental coverage."

### E. AJV / schema validation

#### E1. "Why AJV instead of zod / valibot / joi?"

"AJV consumes JSON Schema directly, which is what OpenAPI uses. Zod is a TypeScript-first DSL — to use it I'd have to transcribe the YAML into TS, which immediately violates 'spec is source of truth.' AJV reads the same YAML the API consumers read; one source, two consumers."

#### E2. "Why allErrors: true and strict: false?"

"allErrors: true so we collect every validation problem in one pass — the response details array is more useful than 'fix this one thing, rerun, fix the next.' strict: false because OpenAPI uses keywords like example, discriminator, nullable that AJV doesn't recognize but should silently allow rather than crash on."

#### E3. "Where exactly are responses validated?"

"Tests assert response shapes via expectSchema(validators.Claim, body) in [tests/support/fixtures.ts:74](tests/support/fixtures.ts:74). The mock validates \*requests\* via validateCreateBody / validateUpdateBody. Both pull from lib/openapi.ts, so spec changes flow to both sides automatically."

#### E4. "Why pre-compile validators at module load?"

"AJV compilation is non-trivial — schema-to-validator-function. Doing it once at startup means every test call is a hot loop. Doing it per request would dominate runtime."

#### E5. "What's formatErrors doing?"

"AJV's raw error objects are unreadable in test failure output — paths in JSON pointer notation, generic messages. formatErrors ([tests/support/schema.ts:34](tests/support/schema.ts:34)) converts them into a readable bullet list with the field path and the issue. When expectSchema fails, the message tells you exactly what was wrong AND dumps the offending value, so debugging takes seconds."

### F. Test framework / Playwright

#### F1. "Why Playwright in API mode? Why not REST Assured, supertest, Postman/Newman, k6?"

"Four reasons. (1) Built-in fixtures, parallel execution, retries, and tracing — all the runner features a serious test suite needs. (2) Same tool covers UI tests later — one stack to teach, one HTML report. (3) webServer block auto-starts the mock — npm test is one command. (4) Tracing on failure gives me an HTTP timeline I can replay. Postman/Newman is fine for collections but bad for code review and reusable abstractions; supertest is in-process which I want sometimes but not for parallel runners; k6 is for load, not contract."

**Follow-up:** *"You sacrifice ecosystem (RestAssured has more matchers)."* → "True for body-level matchers. I'm using AJV for shape and bespoke expect for values, which has covered everything I needed. If I started missing something specific I'd add it as a helper, not switch frameworks."

#### F2. "Walk me through how a single test executes end-to-end."

"Using TC-U1 as the example. (1) Playwright spawns the mock via `webServer` on port 3100. (2) The `claims` fixture in `fixtures.ts` constructs a `ClaimsClient` wrapping Playwright's request context. (3) `aValidCreateClaim()` produces a worker-unique payload. (4) `claims.createOrThrow(payload)` POSTs and parses the response, throwing if not 201. (5) Three sequential PATCH calls walk the claim through `IN_REVIEW` → `APPROVED` (with payout) → `PAID`, each asserting status and on `APPROVED` also schema-validating with `validators.Claim`. (6) On failure, Playwright captures the full HTTP trace which `npx playwright show-trace` replays."

### F3. "Why a `ClaimsClient` class? Why not call `request.post('/claims', ...)` directly in specs?"

"Three things move to one place. (1) URL strings — if `/claims` becomes `/v2/claims`, one file changes. (2) Type-safety — `update(id, body: UpdateClaimRequest)` catches schema mistakes at compile time. (3) Arrange helpers — `createOrThrow` and `advanceThrough` are 'fail-loud' setup primitives. Without them, every spec opens with `expect(res.status()).toBe(201)` boilerplate before the actual assertion. The client itself is 90 lines and reads like a typed SDK for the API."

### F4. "Why `createOrThrow` and `advanceThrough`?"

"They make the \*intent\* of the test obvious. `await claims.createOrThrow(payload)` reads as 'arrange: I need a claim.' If setup fails, the error says 'Arrange step failed: expected 201, got 400, body: {...}' — not a downstream assertion failure that takes you 10 minutes to backtrack. `advanceThrough` is the same idea for sequential PATCH chains. They throw fast, cleanly."

### F5. "Why the data-driven negative matrix in `create-claim.spec.ts`?"

"Two arrays — `missingFields` (4 entries) and `invalidValues` (9 entries). A for loop generates one Playwright `test()` per entry. Adding a new constraint — say `claimantName` max length — is one row in the array. Every entry is a registered test, so the count is honest (13 tests, not 1 with 13 sub-assertions) and failures point at the specific case. The pattern scales — when this hits 50 validation rules, it's still one file."

### F6. "Why retain-on-failure traces?"

"Trace files are large and slow. Capturing every test wastes disk and CI time. `retain-on-failure` only saves them when something's broken — and a broken test is exactly when you want full request/response/timing replay. CI uploads the trace zip on failure so any reviewer can replay without rerunning."

### F7. "How does parallelism work and how do you avoid flakiness?"

"`fullyParallel: true`, capped at 2 workers in CI to avoid saturating shared runners. The mock is shared between workers — if two tests created claims with the same `policyNumber`, list-filter assertions like TC-L2 would be flaky. The fix is in `claim-builder.ts:47`: `policyNumber` encodes `process.pid % 100_000` (worker-unique) plus a per-call counter (test-unique). 100k policies per worker, no shared state, no `/reset` endpoint needed."

**Follow-up:** *"What if `process.pid` recycles across test runs?"* → "It does, but the mock restarts between runs (`reuseExistingServer: false`), so the previous run's data is gone. Worker uniqueness only needs to hold \*within\* a single test run."

### F8. "Why `forbidOnly: !!process.env.CI`?"

"Local dev uses `test.only(...)` to focus on one test. If a developer commits that, CI would silently skip the rest of the suite. `forbidOnly` in CI fails the build, surfacing the mistake before merge."

### F9. "Why retries: 1 in CI? Doesn't that hide flakiness?"

"It does, marginally. The trade-off: most CI flakes are infrastructure (port collision, slow process startup) not test bugs. One retry catches those without hiding real bugs — a test that fails twice almost certainly has a real defect. Local has retries: 0 so flakiness surfaces immediately during development."

**Follow-up:** *"How do you stop retries from masking a regression that fails 50% of the time?"* → "Track flake rate per test. Playwright's HTML report flags retries; if a test flakes more than 3% over a week, it's quarantined and investigated. Don't retry your way out of a real problem."

#### **F10. "Why does test-cases.md exist alongside the tests?"**

"The PDF asks for it (Task 2). It also documents the *intent* in human-readable form — every TC ID maps directly to test ( ' TC-...' ) in the suite. Reviewers can read the catalogue without opening the code, and stakeholders can confirm coverage without grep skills. Cost is keeping it in sync; the TC IDs in test names make drift visible — `grep TC- tests/specs/*.spec.ts` should match the markdown."

#### **F11. "Why fixtures.ts re-exports expect and validators?"**

"Specs only need one import line: `import { test, expect, expectSchema, validators } from '../support/fixtures.js'`; Re-exporting from one module is a small ergonomics win that makes spec files easier to scan."

### **G. Status code semantics**

#### **G1. "Why 400 vs 422? Some say 400 covers everything."**

"I distinguish *\*syntactic/schema\** errors (400) from *\*business-rule\** errors (422). 'policyNumber pattern wrong' is 400 — the request is malformed. 'damageDate is in the future' is 422 — the request is shaped correctly but the domain rejects it. The distinction matters because clients handle them differently: 400 means 'fix your code,' 422 means 'fix your data or workflow.' I'm aware many shops use 400 for everything; both are defensible. The benefit of the split is in the support log — code: `DAMAGE_DATE_IN_FUTURE` is a different ticket type from code: `VALIDATION_ERROR`."

**Follow-up:** *"Some teams use 409 for state-machine violations."* → "Yes — 409 (Conflict) is also defensible for 'this transition collides with current state.' I went with 422 because the underlying RFC defines it as 'syntactically valid but semantically invalid,' which is exactly state-machine + invariant violations. The choice should be a team-wide convention; what matters most is consistency, not the specific code."

#### **G2. "Why 400 (INVALID\_ID) for malformed UUID instead of 404?"**

"404 means 'this resource doesn't exist.' To answer that, the server has to look it up. A non-UUID string isn't 'a missing resource' — it's a malformed request. Returning 404 would also leak existence information: an attacker could probe valid UUIDs by status-code differential. Validating format first means malformed never reaches storage."

#### **G3. "Why does GET /claims with no matches return 200 + [] instead of 404?"**

"Empty result is a normal outcome of a list query, not an error. Forcing 404 makes clients special-case 'no results' vs 'real error,' which is annoying and bug-prone. The list contract is 'an array, possibly empty.' This is TC-L4 — explicit assertion."

#### **G4. "What 5xx codes does your suite test?"**

"None. The mock doesn't have a path that produces 5xx — every error is a deliberate 4xx. In a real service I'd test by injecting failures (kill the DB, kill a downstream) and asserting we return 500/502/503 with a `traceId` rather than leaking stack traces. That's an integration-level concern that needs a different test environment."



## H. CI/CD

### H1. "Walk me through your GitHub Actions workflow."

"Single job, runs on push and PR to main. Checkout → Node 20 with npm cache → `npm ci` → `npm test` (Playwright auto-starts the mock) → upload HTML report on success and failure → upload traces only on failure. `concurrency.cancel-in-progress: true` cancels old runs when a new commit lands on the same branch. Timeout 10 minutes — anything longer is a runaway. About 50 lines."

### H2. "Why upload the report on success too?"

"Two reasons. First, reviewers can see *\*what passed\** — useful when adding tests in a PR, you can read the report directly without rerunning locally. Second, when something passes today and fails tomorrow, having the success report from yesterday makes regression diffing easier."

### H3. "What's missing from this CI?"

"Several things, in priority order. (1) Spec linting — `spectral` or `redocly` to catch OpenAPI mistakes that AJV ignores (missing `description`, inconsistent `operationIds`). (2) Type-checking step — I have `npm run typecheck` but it's not wired into CI. (3) ESLint step — same. (4) Visual diff for the OpenAPI spec on PRs (`oasdiff`) so breaking changes are obvious. (5) Caching the Playwright browsers if I ever add UI tests. (6) Matrix builds across Node 20 and 22 once we pin a target. I cut these for the take-home — happy to add."

### H4. "Why `concurrency.cancel-in-progress`?"

"Saves CI minutes. If I push three commits to a branch in quick succession, the first two runs become stale — the only result that matters is the latest. Cancelling old runs frees runner capacity for active branches."

### H5. "What happens if `npm test` fails halfway through? Are mocked servers cleaned up?"

"Playwright's `webServer` block owns the mock process lifecycle — when the test runner exits, it sends `SIGTERM`. The mock is in-memory so there's no DB to clean up. Worst case: a stuck process holding port 3100, which `MOCK_PORT` env var sidesteps. In CI, the runner is destroyed after the job, so leaked processes are impossible."

## I. Coverage philosophy

### I1. "41 tests for an API with four endpoints. Is that the right amount?"

"It's the amount that gives one reason-to-fail per test. Each TC corresponds to a distinct contract or invariant — happy path, every required field, every pattern constraint, every transition pattern, each business rule. I deliberately *\*didn't\** add tests like 'string with leading whitespace' or 'really long `lossDescription`' because those exercise the same AJV code path as cases I'm already testing. Padding the count adds maintenance cost without catching new bugs."

**Follow-up:** *"What about boundary tests? `minLength`, `maxLength`?"* → "Good catch. `claimantName` `minLength` is tested (TC-C6: `claimantName too short`); `maxLength` isn't. I'd add it. Same for `lossDescription` upper bound and `payoutAmount: 0` (excludes by `minimum: 0.01`). These are clear gaps — if you ask me to add tests, those are the first three."

### I2. "How do you decide when to stop?"

"Three lenses. (1) **Equivalence partitioning** — every distinct *\*behavior path\** has at least one test. (2) **One-bug-one-fail** — each test should be able to fail for a unique reason. (3) **Cost of being wrong** — a state-machine bug in production costs more than a 'unicode emoji in name' bug, so transitions get more density than character-set tests. When the next test I'd write has the same failure mode as one I already

have, I stop."

### 13. "What about negative tests on combinatorial fields?"

"I haven't done full pairwise (no PICT-style combination of `policyNumber` × `claimantName` × `damageDate`). For the take-home, that would be padding because each field validates independently. If validation became cross-field (e.g. `damageDate` must be after `policyStartDate`), pairwise becomes valuable."

### 14. "Where are unit tests?"

"There aren't any — and that's intentional for the scope. The mock is so thin that the API tests *are* the unit tests for it. In a real codebase the production service would have unit tests for the state machine, AJV-driven request validation, and the payout invariants — those tests would run in milliseconds and the API suite would shrink to integration coverage. The mock's lookup-table state machine is too small to merit its own tests."

### 15. "Mutation testing — would you add it?"

"For the production service, yes — `stryker-mutator` against the state machine and invariant code would prove the test suite catches one-character bugs. For a take-home mock, the cost-benefit doesn't work; I'd be testing my own mock's mutations, which proves nothing about the real backend."

## J. Trade-offs and what you cut

### J1. "Tell me about the most important thing you cut."

"Authentication and authorization. The mock is wide-open. In a real claims service, auth is the most security-critical surface — IDOR via `GET /claims/{id}`, role-based PATCH (claim-handler vs approver vs read-only), tenant isolation. I called it out as the first item in 'What else I'd test.' I cut it because the take-home spec didn't define an auth model, and I didn't want to invent one without grounding."

### J2. "How would you actually test auth?"

"Layered. (1) Unauthenticated → 401 on every endpoint. (2) Token-based: expired token → 401, malformed → 401, valid but wrong scope → 403. (3) Role matrix: claim-handler can OPEN → IN\_REVIEW, only approver can APPROVED → PAID, read-only can't PATCH at all. (4) Tenant isolation: tenant A trying to fetch tenant B's claim returns 404 not 403 (avoids existence-probing IDOR). (5) Audit trail — every transition records actor + timestamp."

### J3. "No persistence. Why?"

"In-memory means every CI run starts clean — no test pollution. With a real DB I'd need either ephemeral containers (testcontainers), transactional fixtures (rollback at end of test), or a `/reset` endpoint. All of those are right for a real service; for a mock-only take-home, in-memory is the simpler answer that doesn't sacrifice anything I needed to demonstrate."

### J4. "What changes when persistence is added?"

"(1) Test isolation has to come from somewhere — testcontainers per worker, or transactional rollback, or seeded namespaces. (2) Migration tests become critical (run the migration, run the suite, run the rollback, run the suite again). (3) `createdAt/updatedAt` precision matters — DB-generated timestamps may differ from the request handler's `nowIso()`, so timing assertions like TC-U3 need tolerance. (4) Concurrency tests become real — two PATCHes racing through the state machine need optimistic locking with `If-Match/ETag`."

### J5. "No contract tests with downstream services. Why?"



"There are no downstream services in the take-home spec. In a real claims service I'd want Pact contracts with at least: the policy service (validates `policyNumber`), the payouts service (executes settlement), and the fraud-scoring webhook. Pact stops the accounting team from breaking us on a silent rename — and stops us from breaking them."

## J6. "What WOULD you add tomorrow if you joined the team?"

"Three things in week one. (1) Spec linting in CI (`spectral`) — cheap, catches hygiene drift. (2) `oasdiff` on PRs to flag breaking spec changes loudly. (3) Type-check and lint as required CI steps. Then in week two, identify the most-changed endpoint and write Pact contract tests against the most-coupled consumer."

## K. Insurance domain

### K1. "What's missing from the claims lifecycle that a real claims service would have?"

"Several things I'd push for early. (1) **Reserves** — when a claim is opened, an estimated reserve is set; APPROVED amount may differ. (2) **Partial payments** — APPROVED → PARTIALLY\_PAID → PAID for big claims paid in installments. (3) **Reopening** — REJECTED → IN\_REVIEW under supervisor role within an SLA. (4) **Withdrawal** — OPEN/IN\_REVIEW → WITHDRAWN when the claimant retracts. (5) **Closing without payout** — APPROVED with `payoutAmount: 0` for claims under deductible."

### K2. "What about GDPR / PII?"

"`claimantName` and `lossDescription` are PII. Real tests need to verify (1) PII never appears in structured logs (only `claimId` and `actor` — `traceId` for correlation), (2) right-to-erasure replaces PII with anonymized markers without breaking referential integrity, (3) export endpoints honor data subject access requests, (4) data retention deletes claims past the regulatory window."

### K3. "Multi-tenancy / IDOR?"

"If Emil sells white-label, two tenants must never see each other's claims. Critical tests: tenant A authenticated, requests `GET /claims/{id-from-tenant-B}` → 404 (NOT 403; 403 leaks existence). `GET /claims` returns only tenant A's claims regardless of any client-side filter. PATCH on cross-tenant id → 404. These are usually broken on first implementation."

### K4. "Audit trail?"

"Every transition needs `actor`, `timestamp`, `from_status`, `to_status` in an immutable log. Tests: PATCH a claim, query the audit endpoint, verify the entry. Critical for BaFin and dispute resolution — claims handlers will be asked 'who approved this in 2024?' three years from now."

### K5. "What about money?"

"Three landmines. (1) **Currency mismatch** — never let an EUR claim be paid in USD without an explicit FX step. (2) **Rounding** — payouts must be banker's-rounded to two decimals; floating-point arithmetic in JS is wrong by default. (3) **Fraud thresholds** — payout amounts above a threshold trigger a manual review state, which is its own transition test."

## L. Scaling / what you'd do at Emil

### L1. "If this suite grew to 500 tests, what changes?"

"(1) **Test data** — `aValidCreateClaim` works at small scale; at 500 tests I'd want a richer factory with named scenarios (`aClaimAtApproved()`, `aRejectedClaim()`) so spec files don't repeat the arrange. (2) **Shared mock state** — the in-memory shared mock starts to bottleneck; I'd move to per-worker mock

instances. (3) **Test runtime** — currently fast because everything is in-process; against a real backend, parallelism saturates the service. Need rate-limiting at the test level. (4) **Categorization** — @smoke @regression @slow tags so PR runs can skip the long tail. (5) **Selective execution** — only run tests touching changed endpoints (we'd need impact analysis)."

## L2. "How would you organize the QA function across multiple squads at Emil?"

"Embedded QA in each squad for ownership of feature tests, plus a central QE function that owns the cross-cutting concerns: contract tests, the test framework itself, performance and resilience suites, and CI/CD for tests. Embedded QAs commit to the framework but don't fork it. The central team's KPI is 'how fast can a squad add a new test' — friction there is what pushes squads to skip writing tests."

## L3. "How do you measure QA value?"

"Four signals. (1) **Escaped defects** — bugs found in production per quarter, weighted by severity. (2) **MTTR for test-environment issues** — flakiness, broken envs, port collisions. (3) **Time-to-test for a new feature** — from PR open to test coverage merged. (4) **CI feedback loop** — p50 and p95 of test-run wall time. Pure 'coverage %' is a vanity metric; the four above push toward outcomes."

## L4. "How do you keep tests from rotting?"

"Two policies. (1) **Skipped tests are bugs** — anything `test.skip` has a Jira ticket and a deadline. Quarterly review removes any skip older than the deadline (either fix or delete; never let it fester). (2) **Flake budget per test file** — track retry rate; >3% over a week and the test is quarantined. Both are visible on a dashboard so the team sees rot accumulating."

## M. AI usage (this WILL come up)

### M1. "You used AI to write this. How do you make sure you understand it?"

"Three habits. (1) Before accepting any block of generated code, I make myself write — in the file or in the PR description — *\*why\** this approach over the alternatives. If I can't, the AI is making the decision and I'm rubber-stamping. (2) I read every line and rename or refactor anything that doesn't fit the codebase's voice — generated code has tells, and the goal is one consistent style. (3) For non-trivial choices (the 400/422 split, the 4-pattern transition coverage, in-memory mock vs Prism) I sparred with the AI both ways before deciding. The README's design-decisions section is my paper trail. If you ask me about any line in this repo I can defend it — that's the bar."

### M2. "How do you teach a junior to use AI well on a QA team?"

"Concrete rule: AI generates the boilerplate, you generate the *\*judgment\**. Junior should be able to articulate: what test does this code catch, what would make it fail, what wouldn't it catch. If they can't answer the third question, they pasted without thinking. Pair-test reviews focus on those questions, not the syntax."

### M3. "How do you trust AI-generated code at a team level?"

"(1) Same code review bar as human-authored code — no exceptions. (2) Disclose AI involvement in PR description (Emil's culture seems to encourage this — I'd push for it). (3) Treat AI suggestions as a junior engineer's draft: useful, must be reviewed, accountability stays with the human committing. (4) Watch for failure modes specific to AI — hallucinated APIs, plausible-but-wrong test assertions, security anti-patterns. Add CI checks where possible (lint rules, secret scanners, dependency vulnerability scanners)."

### M4. "What did AI get wrong in your take-home that you fixed?"

"Have at least one honest answer here. Examples to draw from: 'My initial draft had every error as 400 — I argued 400/422 with the AI and decided to split.' / 'AI suggested 15 forbidden-pair tests and I cut it to 4 patterns because the cost-benefit wasn't there.' / 'AI wanted to add full pagination on day one — I cut it because the spec didn't require it and it would have grown the PR.' Pick one that's true for you."

## N. Behavioral / collaboration (Thomas-flavor)

### N1. "Tell me about a disagreement with a developer about test coverage."

Pick a real story. Structure: situation, the disagreement (specific), how you negotiated (data or examples, not authority), the outcome, what you learned. Bonus if you concede the developer was right about \*something\*.

### N2. "How would you onboard a junior QA into this repo?"

"Day 1 — read [README.md](README.md) and [CODEBASE\_GUIDE.md](CODEBASE\_GUIDE.md), run `npm test`, open the HTML report and click into one test. Day 2 — pick one TC ID, write a sentence about what bug it catches, share with me. Day 3 — find a gap (boundary tests, listed in I1) and add one test. By end of week 1, they've shipped a PR. The repo's structure (one spec file per operation, fixtures, factory, client) means there's exactly one place to add anything."

### N3. "How would you work with a PM who wants 'more test coverage' without specifying what?"

"Reframe it. Coverage % is a vanity number. Ask: what's the production incident you're worried about, what's the recent escaped bug we want to never see again, what's the regulatory requirement we need to demonstrate. Translate those into tests. Then track 'time to ship a feature with tests' and 'escaped defect rate' instead — those are the metrics that matter to the PM's actual goals."

### N4. "Tell me about a flaky test you fixed."

Pick a real story. The trick is \*root cause\*, not 'I added a sleep.' Time-based race? Shared state? Network non-determinism? Resource contention? The story should end with the fix preventing the \*category\* of flake, not just that one instance.

### N5. "What's the worst test suite you've inherited and what did you do?"

Real story. Look for: triage by signal value, deletion of dead tests (controversial — they'll watch how you justify), introducing structure (factories, fixtures, naming), CI improvements that surfaced flakes, getting team buy-in for new conventions.

## O. Live extension — most likely asks

Rehearse one of these \*cold\* before the interview. Open the repo, do it end-to-end (spec → mock → tests → green CI). Speed matters — interviewers will lose interest if you fumble the setup.

### O1. "Add a DELETE /claims/{id} (soft delete to status WITHDRAWN)."

Steps:

1. Add WITHDRAWN to ClaimStatus enum in [claims-api.yaml:223](claims-api.yaml:223) and [lib/types.ts:31](lib/types.ts:31).
2. Add OPEN: ['IN\_REVIEW', 'WITHDRAWN'] and IN\_REVIEW: [..., 'WITHDRAWN'] to TRANSITIONS in [src/claims-mock-app.ts:45](src/claims-mock-app.ts:45). (Don't allow withdrawal post-APPROVED — that's a different flow.)
3. Add DELETE /claims/{id} route that PATCHes status to WITHDRAWN (reuse the transition logic).

4. New spec file `tests/specs/delete-claim.spec.ts` — happy path, cannot withdraw from PAID, cannot re-withdraw.

## O2. "Add pagination to GET /claims."

Steps:

1. Add `limit` (default 50, max 200) and `cursor` (opaque) query params to the spec.
2. Mock paginates the sorted array; `cursor` is `claim.createdAt` of last returned item.
3. Tests: default limit, custom limit, cursor traversal returns the next page, beyond-end returns empty.

## O3. "Add an Idempotency-Key header to POST /claims."

Steps:

1. Spec: define Idempotency-Key request header, document semantics.
2. Mock: maintain a `Map<idempotencyKey, claimId>`. Same key + same body → return cached claim. Same key + different body → 409.
3. Tests: replay returns same id, different body returns 409, missing key behaves as before.

## O4. "Find and fix the planted bug" (they may invent one).

The drill: write a *\*failing\** test FIRST, then fix. Do not fix-then-test. Order matters and they're watching for it.

## O5. "Add concurrency tests."

Spawn two parallel PATCHes from `IN_REVIEW` to different terminals (`APPROVED` vs `REJECTED`). Without optimistic locking, both succeed and the second overwrites the first — bug. Tests assert: one wins with 200, one fails with 409, final state is consistent. The mock currently lacks `If-Match/ETag` so this would also be a feature add.

## P. Closing questions YOU ask (pick 3)

- "What's the current contract-testing posture between services here? Pact? Schemathesis? Nothing yet?"
- "Where does QA sit in your SDLC — embedded in squads, central QE function, or hybrid?"
- "What's the biggest test-suite pain point on the team right now: flakiness, runtime, coverage gaps, environment management?"
- "How is the QA function measured? Escaped defect rate? MTTR? Coverage?"
- *\*(For Dmytro)\** "Where do you want AI tooling to be in the engineering workflow 12 months from now — and what guardrails matter most as it gets used more?"
- "What does the first 30 days look like for someone joining this team?"
- *\*(If they ask if you have questions for a specific person)\** — pick the question that matches their seat: tech to Hussein, process/team to Thomas, strategy to Dmytro.

## Final cheat-sheet (memorize these numbers)

- **41** total tests, **4** spec files, **9** TC IDs in create alone
- **5** statuses, **4** legal transitions, terminal states `REJECTED` + `PAID`
- **400** = schema/syntax, **404** = not found, **422** = business rule
- **3100** = Playwright mock port, **3000** = standalone (`npm run start`)
- `MOCK_PORT` env var to override, `BASE_URL` to point at a real server

- LOG\_REQUESTS=true for verbose mock output
- 2 workers in CI, 1 retry in CI, 0 retries locally

**Drill order tonight:** A → C5/C6 → D3 → F3/F4/F5/F7 → G1/G2 → J1/J2 → M1/M3 → O (rehearse one extension end-to-end with a stopwatch).

You've got this. The code is genuinely solid; what they're scoring is whether you OWN it.